

JS Date Object

1. What is the JS Date Object?

- a. The **Date** object in JavaScript is a **built-in** object used to handle and manipulate dates and times.
- b. It allows you to work with current dates, specific dates, and even perform date calculations like finding differences between two dates or formatting them in a readable way.
- c. The **GMT** and **UTC** are two different systems for measuring time.
- d. **GMT** stands for → Greenwich Mean Time
- e. **UTC** stands for → Coordinated Universal Time
- f. Both are time standards, but UTC is the primary global time standard used today, while GMT is a time zone used historically and still in some contexts.

2. How to Create a Date Object?

- a. We can create the Date object with the help of Date Constructor by using **new Date()**.
- b. Most methods simply allow you to **minute** , **second** , and **get** and **set the year** , **month** , **day** , **hour** , **millisecond** fields of the object using either local time or UTC (universal, or GMT time).
- c. **Syntax:**

```
let currentDate = new Date();  
console.log(currentDate); //Fri Sep 05 2025  
15:31:57 GMT+0530 (India Standard Time)
```

3. Example of Current Date and Time (no arguments):

- a. To create a Date object with the current date and time.
- b. With no arguments, the Date() constructor creates a Date object set to the current date and time.
- c. **Syntax:**

 new Date()

4. Example of Specific Date and Time (with arguments):

- a. When one string argument is passed, it is a string representation of a date in the format accepted by the `Date.parse()` method.
- b. **Syntax:**

 new Date(datestring)

- c. Examples of supported formats
 - i. "YYYY-MM-DD" (e.g., "2025-01-12")
 - ii. "YYYY-MM-DDTHH:mm:ss" (e.g., "2025-01-12T15:30:00")
 - iii. "Month Day, Year HH:mm:ss" (e.g., "January 12, 2025 15:30:00")
- d. You can pass individual date and time components as arguments to the `Date` constructor.
- e. It will accept 7 arguments inside the date constructor.
- f. **Custom Date Creation** → new Date("year", "month", "day", "hrs", "min", "sec", "msec");

g. Syntax:



```
new Date(year,month,date,hour,minute,second,millisecond)
```

h. Parameters:

- i. Year → The 4-digit year (e.g., 2025).
 - ii. Month → (Month index 0 for January, 11 for December).
 - iii. Day → Day of the month (1-31).
 - iv. Hour → Hour (0-23).
 - v. Minute → Minute (0-59).
 - vi. Second → Second (0-59).
 - vii. Millisecond → Millisecond (optional)
-
- i. **Return Value** → *It returns the date string containing year, month, day, hours, minutes, seconds, and time zone.*
 - j. **For Example** → `console.log(Date(2025, 9, 4, 15, 46, 34)); //Fri Sep 05 2025 15:46:51 GMT+0530 (India Standard Time)`

5. Properties of the Date Object:

- a. The Date object has several useful properties that provide information about the current time or date.
- b. **Date.now():**
 - i. **Date.now()** returns the number of milliseconds since January 1, 1970.
 - ii. Date.now() is a static method of object.
 - iii. You cannot use it on a date object like **currentDate.now()** (❌)
 - iv. The syntax is always **Date.now()** (✅)

- v. **Return Value:** It will return the timestamp in milliseconds from *January 1, 1970* to till.
- vi. **For Example** → `console.log(Date.now());` // 1757067135195

c. Date.length:

- i. Returns **7**, indicating that the Date constructor accepts up to 7 parameters: *year, month, day, hour, minute, second*, and *millisecond*.
- ii. **For Example** → `console.log(Date.length);` // 7

6. Methods of the Date Object:

- a. The Date object provides numerous methods to *get, set*, and *format date* and time values.

7. Getting Date and Time:

- a. Here are some methods which is used to get the date and time:
 - i. **getFullYear():**
 - 1. Gets the 4-digit year of a date.
 - 2. **Return Value:** A 4-digit number (e.g., 2025).
 - ii. **getMonth():**
 - 1. Gets the month of a date as a number (0 for January to 11 for December).
 - 2. **Return Value:** A number between (0-11).

iii. **getDate():**

1. Gets the day of the month (1-31).
2. **Return Value:** A number between (1-31).

iv. **getDay():**

1. Gets the weekday as a number (0 for Sunday to 6 for Saturday).
2. **Return Value:** A number between (0-6).

v. **getHours():**

1. Gets the hour of a date (in 24-hour format).
2. **Return Value:** A number between (0-23).

vi. **getMinutes():**

1. Get the minutes of a date.
2. **Return Value:** A number between (0-59).

vii. **getSeconds():**

1. Get the seconds of a date.
2. **Return Value:** A number between (0-59).

viii. **getMilliseconds():**

1. Gets the milliseconds of a date.
2. **Return Value:** A number between (0-999).

ix. **getTime():**

1. Gets the total milliseconds since *January 1, 1970* Unix Epoch).

2. **Return Value:** A large number representing the timestamp.

x. **For Example** →

```
// Create a new Date object (current date and time)
let currentDate = new Date();

console.log("Full Year:", currentDate.getFullYear()); // 2025
console.log("Month:", currentDate.getMonth()); // 8
console.log("Date:", currentDate.getDate()); // 5
console.log("Day (Weekday):", currentDate.getDay()); // 5
console.log("Hours:", currentDate.getHours()); // 15
console.log("Minutes:", currentDate.getMinutes()); // 53
console.log("Seconds:", currentDate.getSeconds()); // 56
console.log("Milliseconds:",
currentDate.getMilliseconds()); // 72
console.log("Timestamp (ms):", currentDate.getTime()); // 1757067850025
```

8. Setting Date and Time:

a. Here are some methods which is used to get the date and time:

i. **setFullYear(year, [month], [day]):**

1. Sets the year (and optionally the month and day) of a date.
2. **Return Value:** The timestamp (in milliseconds) of the updated date.

ii. **setMonth(month, [day]):**

1. Sets the month 0 for January to 11 for December) and optionally the day.
2. **Return Value:** The timestamp (in milliseconds) of the updated date.

iii. **setDate(day):**

1. Sets the day of the month 1-31.
2. **Return Value:** The timestamp (in milliseconds) of the updated date.

iv. **setHours(hours, [minutes], [seconds], [milliseconds]):**

1. Sets the hour 0-23 and optionally the minutes, seconds, and milliseconds.
2. **Return Value:** The timestamp (in milliseconds) of the updated date.

v. **setMinutes(minutes, [seconds], [milliseconds]):**

1. Sets the minutes and optionally the seconds and milliseconds.
2. **Return Value:** The timestamp (in milliseconds) of the updated date.

vi. **setSeconds(seconds, [milliseconds]):**

1. Sets the seconds and optionally the milliseconds.
2. **Return Value:** The timestamp (in milliseconds) of the updated date.

vii. setMilliseconds(milliseconds):

1. Sets the milliseconds 0-999 of the date.
2. **Return Value:** The timestamp (in milliseconds) of the updated date.

viii. setTime(milliseconds):

1. Sets the entire date and time based on the number of milliseconds since *January 1, 1970* Unix Epoch).
2. **Return Value:** The timestamp (in milliseconds) of the updated date.

For Example →

```
// Create a new Date object
let date = new Date();

console.log("Original Date:", date);
// Original Date: Fri Sep 05 2025 16:01:20 GMT+0530
(India Standard Time)

// setFullYear(year, [month], [day])
date.setFullYear(2030, 5, 15); // Sets year = 2030, month
= June (5), day = 15
console.log("After setFullYear:", date);
// After setFullYear: Sat Jun 15 2030 16:01:41 GMT+0530
(India Standard Time)

// setMonth(month, [day])
date.setMonth(10, 25); // Sets month = November (10), day
= 25
console.log("After setMonth:", date);
// After setMonth: Mon Nov 25 2030 16:01:53 GMT+0530
(India Standard Time)

// setDate(day)
date.setDate(5); // Sets day = 5
console.log("After setDate:", date);
// After setDate: Tue Nov 05 2030 16:02:27 GMT+0530
(India Standard Time)

// setHours(hours, [minutes], [seconds], [milliseconds])
date.setHours(14, 30, 45, 500); // 14:30:45.500
console.log("After setHours:", date);
```

```
// After setHours: Tue Nov 05 2030 14:30:45 GMT+0530
(India Standard Time)

// setMinutes(minutes, [seconds], [milliseconds])
date.setMinutes(50, 20, 900); // 50 min, 20 sec, 900 ms
console.log("After setMinutes:", date);
// After setMinutes: Tue Nov 05 2030 14:50:20 GMT+0530
(India Standard Time)

// setSeconds(seconds, [milliseconds])
date.setSeconds(10, 250); // 10 sec, 250 ms
console.log("After setSeconds:", date);
// After setSeconds: Tue Nov 05 2030 14:50:10 GMT+0530
(India Standard Time)

// setMilliseconds(milliseconds)
date.setMilliseconds(750); // 750 ms
console.log("After setMilliseconds:", date);
// After setMilliseconds: Tue Nov 05 2030 14:50:10
GMT+0530 (India Standard Time)

// setTime(milliseconds since Jan 1, 1970)
date.setTime(1700000000000);
console.log("After setTime:", date);
// After setTime: Wed Nov 15 2023 03:43:20 GMT+0530
(India Standard Time)
```